Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №1
по дисциплине
«Интернет-технологии и веб-программирование»
Вариант 100

Выполнила: ст. гр. 320604
К. А. Хаджинова

Проверила: Н. В. Хаджинова

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Обзор предметной области 4
2.2 Теоретические сведения 4
2.3 Настройка среды разработки и структуры проекта 5
2.4 Реализация и тестирование эндпоинтов 6
3 Ответы на контрольные вопросы 11
Выводы 14
Приложение А (обязательное) Листинг файла *server.js* 15

1 ЦЕЛЬ РАБОТЫ

Приобретение практических навыков создания серверного приложения на платформе *Node.js* с использованием фреймворка *Express.js*. Изучение основ маршрутизации, обработки *HTTP*-запросов различных типов (*GET*, *POST*, *PUT*, *DELETE*) и формирования ответов сервера для последующей интеграции с клиентской частью курсового проекта.

2 ХОД РАБОТЫ

2.1 Обзор предметной области

В качестве предметной области выбран сервис *DiagramCode* — веб-приложение для описания и визуализации диаграмм на собственном текстовом языке (*DSL*): *UML*, *BPMN*, *ERD*, сети Петри, *IDEF*0/*IDEF*3, *DFD*. Подобные системы востребованы там, где диаграмму удобнее хранить и версионировать как текст, а не как графический файл: в документации распределённых систем, при проектировании бизнес-процессов, в учебных курсах по моделированию.

Основная информационная сущность приложения — «Диаграмма» (*Diagram*). Согласно реализованной модели данных, каждая диаграмма характеризуется атрибутами:

– *id* – числовой идентификатор, автоматически увеличивающийся (*autoincrement*) при создании новой записи в памяти сервера.

– *title* – название диаграммы, обязательное поле.

– *notation* – нотация диаграммы (например, *erd*, *bpmn*, *uml*), обязательное поле.

– *dslContent* – текстовое *DSL*-описание диаграммы; необязательное поле, по умолчанию пустая строка.

– *createdAt* – дата и время создания записи в формате *ISO*, проставляется автоматически.

Архитектура построена по модульному принципу с разделением на маршруты (*routes*), контроллеры (*controllers*) и модели (*models*). Все запросы к ресурсу диаграмм обрабатываются по базовому *URL*-пути /*diagrams*.

2.2 Теоретические сведения

Node.js – программная платформа на движке *V*8 (*Google*), выполняющая *JavaScript*-код вне браузера, на стороне сервера. Ключевые архитектурные особенности -- модульная система (*CommonJS*: *require*/*module.exports*) и асинхронная, событийно-ориентированная модель выполнения на едином потоке.

Express.js -- минималистичный фреймворк поверх *Node.js* для построения веб-серверов. Приложение создаётся вызовом функции *express*(), которая возвращает объект *app* с методами конфигурации, запуска (*app.listen*()) и маршрутизации *HTTP*-запросов.

Центральные концепции *Express* -- маршрутизация (*routing*: связка *URL*

+ *HTTP*-метод с функцией-обработчиком) и *middleware* -- функции вида (*req*, *res*, *next*), выполняющиеся последовательно в порядке регистрации через *app.use*(); обработчики ошибок -- их частный случай с сигнатурой (*err*, *req*, *res*, *next*).

2.3 Настройка среды разработки и структуры проекта

В качестве *IDE* использовался *VS Code*; для работы сервера установлены *Node.js* (*LTS*) и *npm*. (см. рисунок 2.1).



Рисунок 2.1 – Проверка установленных версий *Node.js* и *npm*

Проект организован по слоям и включает точку входа и четыре специализированные директории. Корневой файл *server.js* является точкой входа в приложение: он создаёт экземпляр сервера через фабричную функцию *createApp*() и запускает прослушивание сетевого порта вызовом *app.listen*(*port*). Сама сборка *Express*-приложения вынесена в отдельный файл *app.js*, который подключает встроенный *middleware express.json*() для разбора тела входящих запросов, монтирует маршруты ресурса диаграмм по базовому пути */diagrams*, а также регистрирует последними два обработчика -- *notFoundHandler* для несуществующих маршрутов и *errorHandler* для прочих ошибок; такое разделение на *server.js* и *app.js* сделано намеренно, чтобы собранное приложение можно было использовать в тестах без реального прослушивания порта (см. рисунок 2.2).

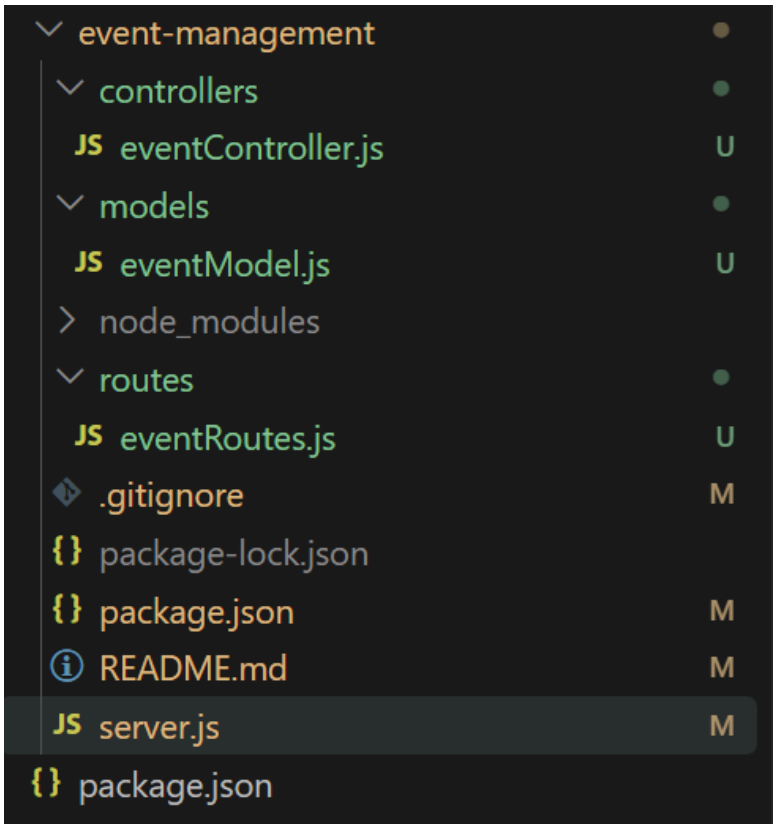


Рисунок 2.2 – Структура проекта *EventManager*

Директория *routes* содержит файл *diagramRoutes.js*, который связывает *HTTP*-методы и *URL*-пути с соответствующими функциями контроллера, не описывая при этом никакой бизнес-логики самостоятельно. Директория *controllers* содержит файл *diagramController.js* -- слой бизнес-логики, в котором реализованы пять *CRUD*-функций (*getAll*, *getById*, *create*, *update*, *remove*) и вспомогательная функция *validatePayload*, проверяющая, что обязательные поля *title* и *notation* присутствуют и являются непустыми строками. Директория *models* содержит файл *diagramStore.js* -- слой представления данных, реализующий хранилище диаграмм в виде массива в оперативной памяти процесса с функциями *findAll*, *findById*, *create*, *update* и *remove*. Отдельная директория *middleware* содержит файл *errorHandler.js* с двумя обработчиками: первый возвращает статус 404 для любого маршрута, не описанного в приложении, второй перехватывает ошибки, переданные через *next*(*err*), и возвращает статус 500. Наконец, директория *tests* содержит автоматические тесты, написанные с использованием *Jest* и *Supertest*, которые проверяют работу всех пяти маршрутов и по итогу дают стопроцентное покрытие кода.

2.4 Реализация и тестирование эндпоинтов

Для взаимодействия с коллекцией диаграмм был спроектирован и реализован *REST API*, состоящий из пяти маршрутов. Таблица 2.1 обобщает используемые *HTTP*-методы, ожидаемые статусы ответов и примеры передаваемых и возвращаемых данных.

Таблица 2.1 – Перечень маршрутов и методов *API*

Маршрут	Метод	Ожидаемый статус	*JSON*	Пример ответа (*JSON)*
/*events*	*GET*	200 *OK*	*(*Отсутствует*)*	[{"*id*":"1", "*title*":"Конференция", "*type*":"*conference*", "*date*":"2026-09-09", "*description*":""}]
/*events*/:*id*	*GET*	200 *OK* / 404 *Not Found*	*(*Отсутствует*)*	{"*id*":"1", "*title*":"Конференция", "*type*":"*conference*", "*date*":"2026-09-09", "*description*":""}
/*events*	*POST*	201 *Created* / 400 *Bad Request*	{"*title*":"Новый вебинар", "*type*":"*webinar*", "*date*":"2026-10-10"}	{"*id*":"1694...5", "*title*":"Новый вебинар", "*type*":"*webinar*", "*date*":"2026-10-10", "*description*":""}
/*events*/:*id*	*PUT*	200 *OK* / 400 / 404	{"*title*":"Обновленная встреча", "*type*":"*meeting*", "*date*":"2026-11-11"}	{"*id*":"1", "*title*":"Обновленная встреча", "*type*":"*meeting*", "*date*":"2026-11-11", "*description*":""}
/*events*/:*id*	*DELETE*	204 *No Content* / 404	*(*Отсутствует*)*	*(*Пустое тело ответа*)*

Для проверки работоспособности *REST API* использовалась среда *Postman*. Ниже подробно описано тестирование каждого эндпоинта для базовых операций *CRUD*.

Операция чтения всей коллекции, реализованная посредством *HTTP*-метода *GET* по адресу /*diagrams*, предназначена для получения массива всех диаграмм, которые на текущий момент хранятся в памяти сервера.

Запрос выполняется без передачи данных в теле, контроллер *getAll* просто вызывает *store.findAll*() и сериализует полученный массив в *JSON*; в случае успешной обработки сервер всегда возвращает статус 200 *OK*, поскольку даже пустая коллекция диаграмм считается корректным, не ошибочным результатом (см. рисунок 2.3).

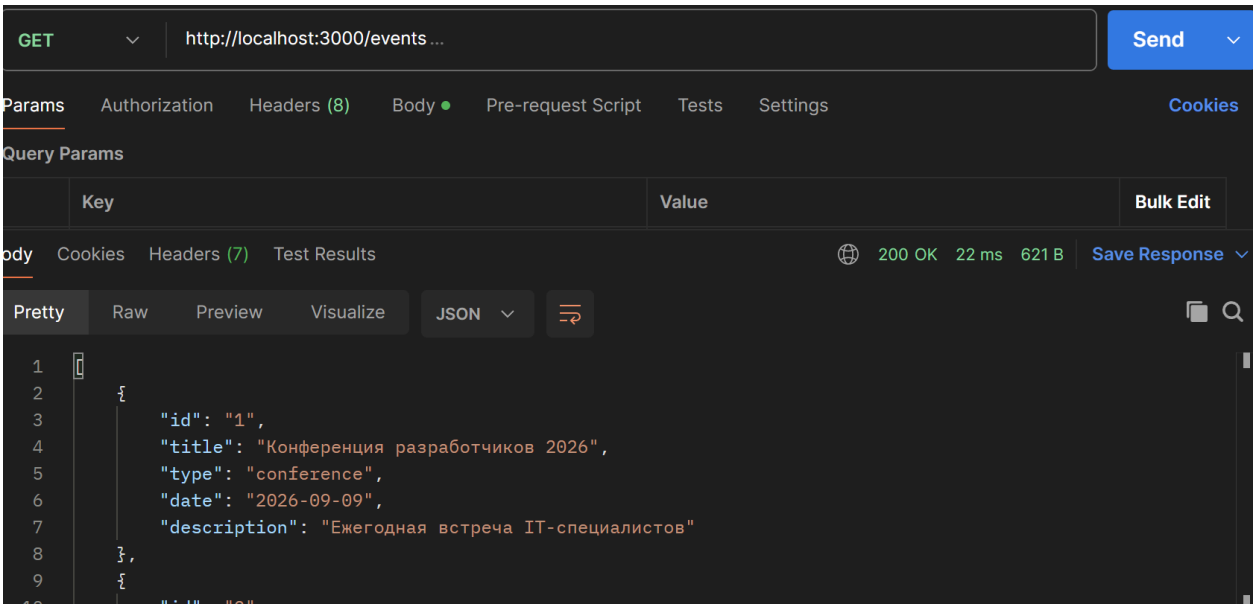


Рисунок 2.3 – Результат выполнения *GET*-запроса

Операция создания реализуется через *HTTP*-метод *POST* с передачей данных новой диаграммы в теле запроса в формате *JSON*. Перед сохранением контроллер *create* вызывает *validatePayload*, которая проверяет, что поля *title* и *notation* присутствуют и являются непустыми строками; если хотя бы одно из условий не выполнено, сервер немедленно возвращает статус 400 *Bad Request* с текстом ошибки, не обращаясь к хранилищу вовсе. При успешной валидации *store.create* формирует новый объект диаграммы, присваивает ему автоматически увеличивающийся числовой *id*, подставляет пустую строку для необязательного поля *dslContent*, если оно не передано, проставляет текущую дату и время в поле *createdAt* и добавляет запись в массив; клиенту возвращается статус 201 *Created* вместе с созданным объектом (см. рисунок 2.4).

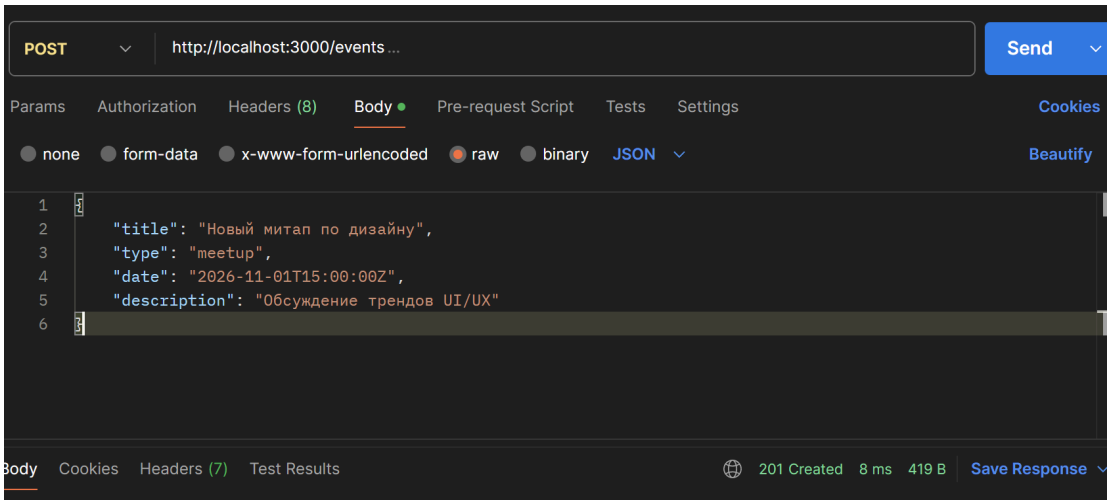


Рисунок 2.4 – Результат выполнения *POST*-запроса

Операция получения одной диаграммы по идентификатору выполняется посредством *GET*-запроса к эндпоинту */diagrams/:id*. Контроллер *getById* извлекает переданный в *URL* параметр через *req.params.id*, приводит его к числу и ищет соответствующий объект в массиве с помощью *store.findById*. Если диаграмма с таким *id* существует, сервер возвращает её в теле ответа со статусом 200 *OK*; если диаграмма не найдена -- явно возвращается статус 404 *Not Found* с телом {"*error*":"Диаграмма не найдена"}, а не пустой либо неопределённый результат, что важно для предсказуемого поведения клиента (см. рисунок 2.5).

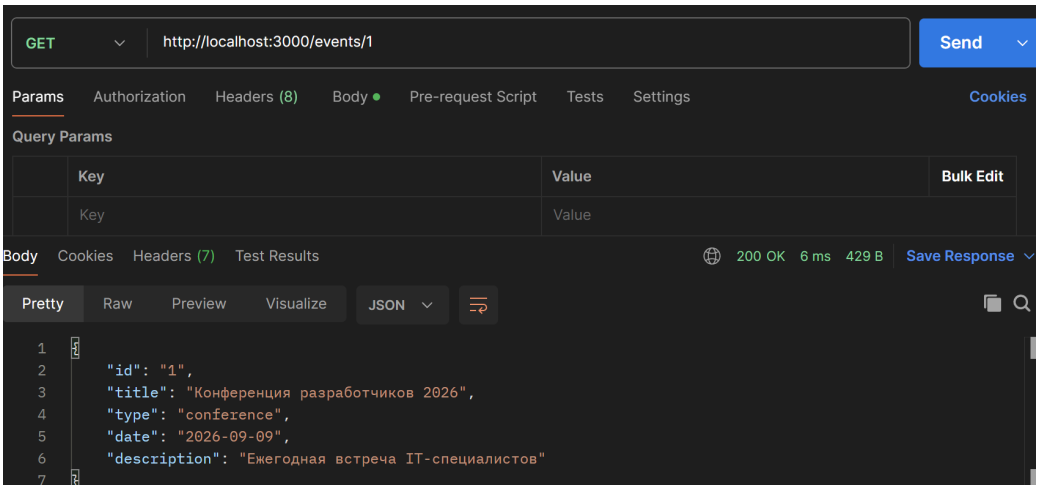


Рисунок 2.5 – Результат выполнения *GET*-запроса по *ID*

Операция обновления реализуется через *HTTP*-метод *PUT* и предназначена для полной перезаписи существующего ресурса по переданному идентификатору. Так же, как и при создании, входные данные

сперва проходят через *validatePayload* -- обновлённые *title* и *notation* обязательны, поскольку *PUT* подразумевает замену ресурса целиком, а не частичное изменение отдельных полей. Если валидация не пройдена, возвращается 400 *Bad Request*; если диаграмма с указанным *id* не найдена в хранилище, контроллер возвращает 404 *Not Found*; при успешном выполнении обоих условий *store.update* перезаписывает поля найденного объекта и возвращает его с статусом 200 *OK* (см. рисунок 2.6). Оба сценария -- успешное обновление и попытка обновить несуществующую диаграмму -- были протестированы отдельно (см. рисунок 2.7).

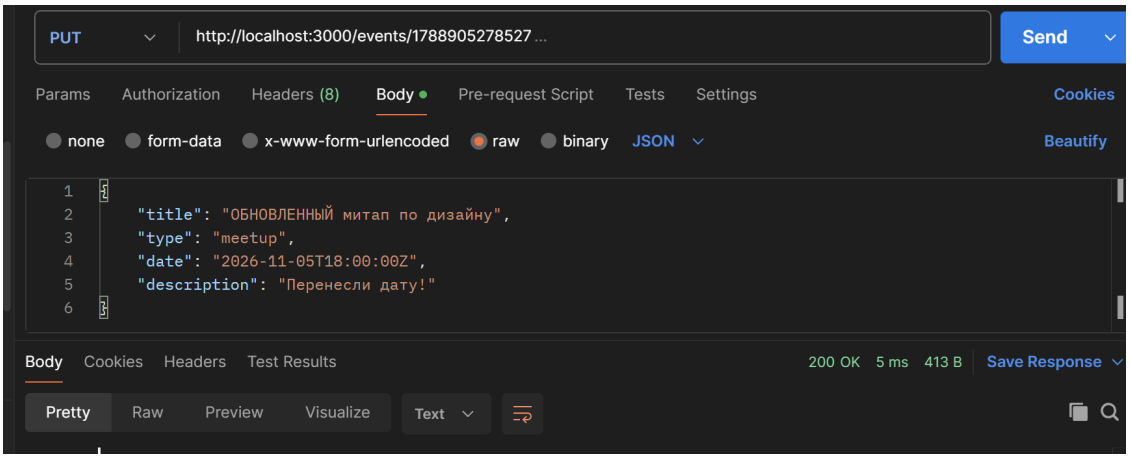


Рисунок 2.6 – Результат выполнения *PUT*-запроса

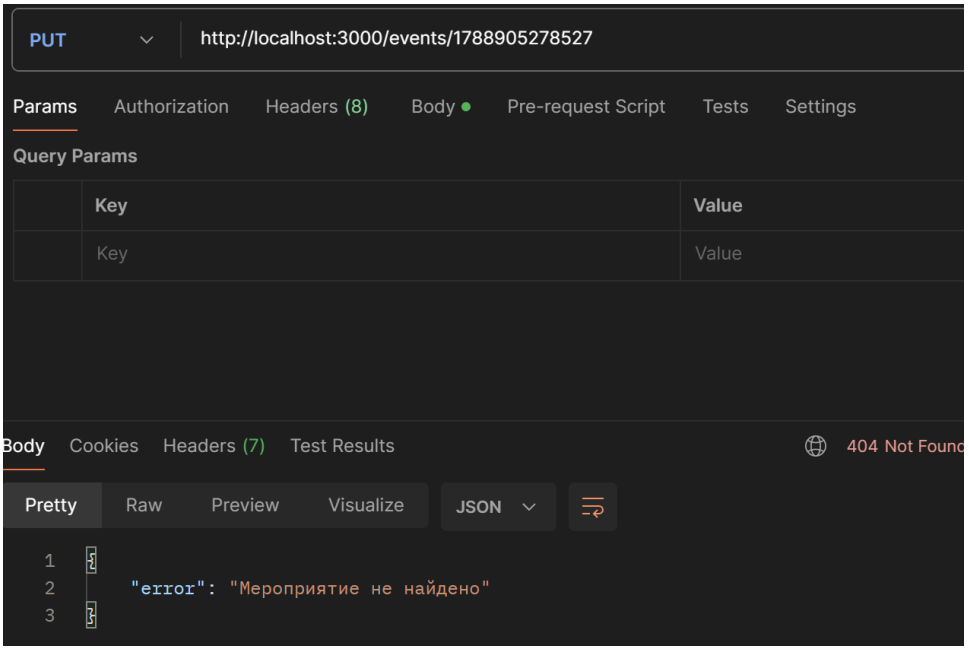


Рисунок 2.7 – Результат выполнения *PUT*-запроса
404 *Not Found*

Операция удаления реализуется через *HTTP*-метод *DELETE* и предназначена для удаления диаграммы из хранилища по указанному идентификатору. Контроллер *remove* вызывает *store.remove*, которая ищет индекс записи в массиве и, если запись найдена, удаляет её методом *splice* и возвращает *true*; в этом случае сервер отвечает статусом 204 *No Content* без какого-либо тела ответа, поскольку удалённый ресурс уже не существует и возвращать нечего. Если запись с таким *id* не найдена, *store.remove* возвращает *false*, и контроллер отвечает статусом 404 *Not Found*. Отдельно был протестирован сценарий повторного удаления уже удалённой диаграммы -- сервер корректно возвращает 404 вместо повторного 204, что подтверждает отсутствие ложного успеха при отсутствующем ресурсе (см. рисунок 2.8).

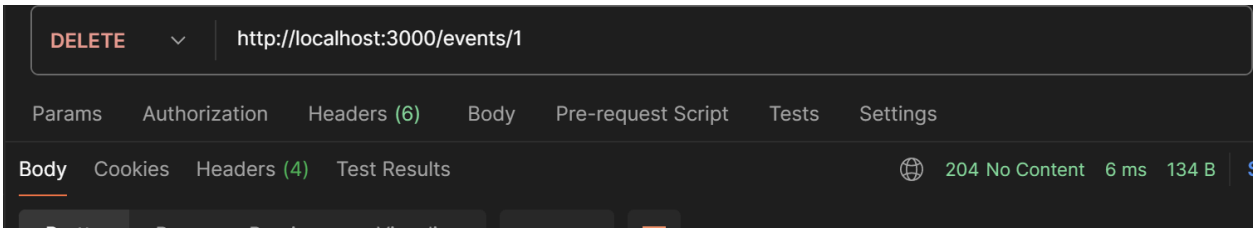


Рисунок 2.8 – Результат выполнения *DELETE*-запроса

Ссылка на репозиторий: *[https://github.com/KsushaKhadzhinova/dsl-app/tree/lab21]*

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какие методы *HTTP* вы реализовали и для каких операций?

Были реализованы методы *GET* для получения списка всех диаграмм и одной по *id*, *POST* для создания новой диаграммы, *PUT* для полного обновления данных существующей диаграммы и *DELETE* для удаления диаграммы.

2. Как получить параметры из *URL* (*route params*) и строку запроса (*query string*) в *Express*?

Параметры из пути *URL* (например, *id* в маршруте /*diagrams*/:*id*) извлекаются через объект *req.params.id*. Данные из строки запроса (то, что передаётся после знака вопроса, например ?*sort=title*) извлекаются через объект *req.query*.

3. Для чего используется *middleware express.json*()?

Этот встроенный *middleware* необходим для автоматического разбора входящих *HTTP*-запросов, заголовок *Content-Type* которых установлен как *application/json*. Он преобразует *JSON*-строку в объект *JavaScript*, доступный в коде через *req.body*.

4. Какой *HTTP*-статус возвращается при успешном создании ресурса? При успешном удалении?

При успешном создании ресурса (*POST*) возвращается статус 201 *Created*. При успешном удалении (*DELETE*) возвращается статус 204 *No Content* (сервер успешно обработал запрос, но в ответе не передаёт никакого содержимого).

5. Что такое *REST API*? Перечислите основные принципы *REST*.

REST API -- это архитектурный стиль взаимодействия компонентов распределённого приложения по протоколу *HTTP*. Основные принципы: клиент-серверная архитектура (разделение фронтенда и бэкенда), отсутствие состояния на сервере (*Stateless* -- каждый запрос содержит все необходимые данные), кэшируемость ответов, единообразие интерфейса (использование стандартных *HTTP*-методов и *URI*) и многоуровневая система.

6. Как организовать глобальную обработку ошибок в *Express*?

Для этого создаётся специальная функция *middleware*, принимающая ровно 4 аргумента: (*err*, *req*, *res*, *next*). В данном проекте это функция

errorHandler, зарегистрированная через *app.use*() в самом конце файла *app.js*, после всех маршрутов и после *notFoundHandler*.

7. Какую структуру ответа сервера вы выбрали для ошибок?

Для ответа при возникновении ошибок используется стандартизированный *JSON*-объект с одним полем *error*, содержащим текстовое описание проблемы.

8. Как протестировать *DELETE*-запрос, если ресурс уже был удален? Какой статус должен быть возвращен?

Необходимо повторно отправить *DELETE*-запрос на тот же *URL* (например, /*diagrams*/1) через *Postman*. Поскольку ресурса в массиве больше нет, сервер не сможет его найти и должен вернуть статус 404 *Not Found*.

9. Какие преимущества дает использование *nodemon* в процессе разработки?

Nodemon автоматически отслеживает любые изменения в файлах проекта и перезапускает *Node.js* сервер. Это избавляет разработчика от необходимости вручную останавливать и заново запускать сервер после каждого изменения кода.

10. В чем разница между методами *PUT* и *PATCH*? Какой из них вы реализовали и почему?

Метод *PUT* используется для полной замены существующего ресурса новыми данными. Метод *PATCH* используется для частичного изменения ресурса (передаются только те поля, которые нужно обновить). В данной работе реализован метод *PUT*, так как по заданию требовалось реализовать операцию полного обновления диаграммы.

11. Как можно добавить поддержку *CORS* в ваше приложение? (Теоретически)

Необходимо установить пакет *cors* командой *npm install cors*, затем импортировать его в *app.js* (*const cors = require*('*cors*');) и подключить как глобальный *middleware*: *app.use*(*cors*());. Это разрешит браузерам делать запросы к *API* с других доменов.

12. Почему в данном задании данные хранятся в массиве, а не в базе данных? Когда стоит переходить на БД?

Хранение в массиве используется на начальном этапе для изучения

основ маршрутизации и принципов работы *HTTP*. На полноценную БД стоит переходить, когда требуется надёжное хранение данных, которое не будет удаляться при перезагрузке сервера, а также для работы со сложными связями и большими объёмами информации -- это и реализовано в следующей лабораторной работе через *Sequelize* и *PostgreSQL*.

13. Какой заголовок *Content-Type* должен быть установлен для отправки *JSON*-данных?

Должен быть установлен заголовок *Content-Type*: *application/json*.

14. Напишите пример *curl*-команды для тестирования вашего *POST*-эндпоинта.

curl -X POST http://localhost:3000/*diagrams -H* "*Content-Type*: *application/json*" *-d* "{"*title*":"Схема заказов","*notation*":"*erd*"}"

15. Как убедиться, что сервер работает корректно после каждой модификации кода?

Следует посмотреть в консоль терминала на предмет отсутствия синтаксических ошибок (*nodemon* выведет сообщение о перезапуске сервера) и выполнить соответствующие запросы через *Postman*, чтобы убедиться, что сервер возвращает ожидаемые статус-коды и *JSON*-данные.

16. Какие шаги нужно предпринять, чтобы запустить ваше приложение на другом компьютере?

Скопировать файлы проекта (обязательно исключив тяжёлую папку *node_modules*). На новом компьютере открыть проект в терминале, выполнить команду *npm install* (она прочитает *package.json* и скачает все нужные библиотеки), после чего запустить сервер командой *npm run dev* или *npm start*.

ВЫВОДЫ

В результате выполнения лабораторной работы разработано серверное *REST API*-приложение «*DiagramCode*» на платформе *Node.js* с использованием фреймворка *Express.js* для управления коллекцией диаграмм. Реализованы маршруты для всех базовых операций *CRUD*, валидация входных данных на уровне контроллера и единообразная обработка ошибок при обращении к несуществующим ресурсам и маршрутам.

На практике освоены ключевые инструменты современного веб-разработчика: интегрированная среда разработки *VS Code*, фреймворк *Express.js* поверх *Node.js* и система контроля версий *Git*. Выполнен полный цикл работы с кодом: от локальной инициализации репозитория и фиксации изменений до публикации исходного кода на платформе *GitHub*.

Кроме того, закреплены навыки управления программными модулями с помощью пакетного менеджера *npm* и организации приложения по слоям (*routes*/*controllers*/*models*/*middleware*). Реализованное серверное приложение протестировано через *Postman* по всем пяти маршрутам и покрыто автоматическими тестами (*Jest* + *Supertest*, 100% покрытие), что создаёт надёжную техническую базу для дальнейшего перехода на реляционную базу данных в лабораторной работе №2.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг файла *app.js*

```
// app.js
const express = require("express");
const diagramRoutes = require("./routes/diagramRoutes");
const { notFoundHandler, errorHandler } = require("./middleware/errorHandler");

function createApp() {
  const app = express();
  app.use(express.json());
  app.use("/diagrams", diagramRoutes);
  app.use(notFoundHandler);
  app.use(errorHandler);
  return app;
}

module.exports = createApp;
```

Листинг файла *server.js*

```
// server.js
const createApp = require("./app");

const port = process.env.PORT || 3000;
const app = createApp();

app.listen(port, () => console.log(`Server running on port ${port}`));
```